The Significance and Effectiveness of Implementing Human Natural Structure (HNS) in Hardware

A Hardware-Embedded Architecture for Independent, Model-Agnostic and Tamper-Resistant AI Safety Verification

Satoru Hara — Natural Structure Works (NSW), 2026
github.com/satoru-hara/03_NSW · naturalstructureworks.com

Audience note: This report is written for readers familiar with AI safety, standardization, and hardware security. It assumes that absolute safety of an autonomous system is unattainable, and therefore states each of its three claimed properties — independence, model-agnosticism, tamper-resistance — with an explicit, bounded definition (§3).

---

0. Executive Summary

Current AI safety largely depends on mechanisms the model vendor defines, implements, and can alter — an arrangement that is difficult to verify from outside and straightforward to bypass. This report sets out the **significance** and **effectiveness** of implementing the Human Natural Structure (HNS) coordinate system in hardware (ROM, FPGA, or ASIC) as the basis for a safety-verification layer with three properties:

- **Independent** — verification performed by an external layer that does not rely on the model's or vendor's self-assessment and cannot be altered by the system under audit.
- **Model-Agnostic** — applicable to any model, because it operates on outputs externally and requires no access to or modification of model internals; it survives model updates and retraining.
- **Tamper-Resistant** — its enforcement and logging are physically immutable: the check cannot be silently disabled and its records cannot be altered after the fact.

These properties concern the verification *apparatus*, not the *correctness* of every judgment: attribution and scoring of an output remain model-dependent (§3), and coverage is bounded to risks expressible in HNS coordinates. Within that scope, hardware embedding converts safety from a matter of vendor policy into a property of the system's architecture.

One question is left explicitly open. Hardware-mediated checking may impose a significant latency/throughput cost; its size is unknown and can only be settled empirically. This report therefore presents the architecture as a hypothesis to be tested by a proof of concept, and §8 treats performance as the decisive open variable.

1. Introduction: The Unresolved Challenges of AI Safety

1.1 Limitations of current approaches

Three limitations recur in current AI safety practice:

- **Vendor-defined internal mechanisms** — safety behaviour is specified and controlled by the same party that builds and operates the model.
- **Bypass and jailbreak** — internal guardrails can be circumvented through adversarial prompting or configuration.
- **Limited external auditability** — there is often no independent, durable record that a safety check ran and produced a given result.

1.2 Why this matters

Comprehensive, decision-by-decision human verification of an autonomous system does not scale. A practical regime must instead make a defined subset of checks *dependable* — applied without exception, impossible to disable quietly, and recorded in a way an outside party can trust. That is an architectural problem, and it is the problem this report addresses.

1.3 The role of HNS

HNS supplies a finite, structured coordinate system against which AI outputs can be mapped and checked externally. It does not replace model-internal safety; it provides an independent layer above it, grounded in a standardized structural schema rather than vendor-specific rules.

2. Human Natural Structure (HNS) in Brief

HNS-36 is a finite, closed, machine-readable coordinate system of 36 cells (6 natural layers × 6 cognitive categories). Each AI output subject to verification is attributed to a coordinate, and structural deviations (for example, an unexplained cross-layer causal jump) are flagged against that schema. The External Verification Architecture (EVA) performs this attribution and writes structured audit records; the External Control System (ECS) acts on the result — deliver, flag, hold, or block.

Scoping premise (stated, not hidden): attribution and deviation scoring are semantic judgments and are model-dependent. This report treats that judgment engine as given. Its claims concern the *layer around* the judgment — how it is applied, why it is independent of the audited system, why it works with any model, and why it cannot be disabled or rewritten — not the correctness of each individual verdict.

3. Definitions and Scope of the Three Properties

Because the title's three properties are strong words, each is defined here precisely, so that the claims are bounded and verifiable rather than rhetorical.

3.1 Independent

Independent means architectural independence of the verification layer from the system it audits: it operates as a sidecar, separate from the model; it does not rely on the model's or vendor's self-assessment; and it cannot be modified or disabled by the audited system. It does **not** mean that the judgment is computed without any model-like process — the attribution engine may itself be learned. Independence here is of the *apparatus and its records*, in the

sense in which an external auditor is independent.

3.2 Model-Agnostic

Model-agnostic means applicability to any AI model — LLM, multimodal, or future systems — because the layer operates on outputs at the interface and requires no access to, or modification of, model weights, architecture, or training. A consequence is durability: the layer survives model updates and retraining. It does **not** mean that detection quality is identical across all models or output types; that is an empirical property to be measured.

3.3 Tamper-Resistant

Tamper-resistant means that, where the hardware is present in the enforcement path, the check cannot be silently disabled by software, vendor, or operator; the enforcement action on a verdict cannot be suppressed by software state; and audit records are write-once / append-only and cannot be altered after writing. It does **not** mean the layer cannot be *omitted* in an uncertified deployment (§7, O4) — tamper-resistance protects against modification, not against a deployer who never installs it.

4. Significance: Why Hardware Embedding Matters

4.1 From vendor policy to architectural property

When the three properties above are implemented in software, they remain promises that a vendor can change. Embedding them in hardware (ROM/FPGA/ASIC) changes their status: the safety check becomes a fixed part of the output path, its enforcement a physical action, and its logs physically immutable. Safety thereby shifts from "what the vendor says it does" to "what the architecture structurally enforces."

4.2 An external trust anchor

Hardware embedding gives an independent auditor something concrete to rely on: a check that demonstrably ran, an action that demonstrably followed, and a record that demonstrably was not altered — none of which depend on trusting the model's operator. This applies the established principle of hardware roots of trust to AI safety verification.

4.3 Regulatory relevance (context, not conformity)

The properties map onto recognized requirements — EU AI Act logging/record-keeping (Art. 12) and human oversight (Art. 14), ISO/IEC 42001 and 23894 governance and risk management, and ENISA's hardware-based trust anchors. This is noted as relevance, not as a claim of certified conformity.

5. Effectiveness: What the Architecture Achieves

5.1 Independent verification

Because EVA sits external to the model and writes its own records, a third party can verify safety outcomes without access to model internals and without trusting vendor self-assessment.

The verification does not share fate with the system it checks.

5.2 A model-agnostic safety layer

The same layer applies to any model that produces outputs, and continues to apply after the model is updated or retrained. This makes the safety investment durable and portable across an AI estate, rather than tied to one model generation.

5.3 Tamper-resistant enforcement and audit

In a deployment that includes the hardware in-path, the safety check cannot be quietly switched off, its blocking or holding action cannot be suppressed by software, and its audit trail cannot be edited after the fact. This yields a dependable, non-bypassable floor for the bounded class of risks the schema covers.

6. Architecture and Workflow

6.1 Implementation options

- **ROM** — maximal immutability for the fixed schema and log anchoring; inflexible.
- **FPGA** — flexible; suited to prototyping and to measuring real-world latency.
- **ASIC** — steady-state optimization at scale, warranted once value and cost are demonstrated.

6.2 External verification workflow

1. The AI system produces an output; no modification of the model is required.
2. EVA receives the output at the interface and attributes it to an HNS coordinate.
3. EVA scores structural deviation and writes an immutable audit record.
4. ECS applies the control policy — deliver, flag, hold, or block.
5. On hold or block, a human operator is notified; the record is available for independent audit.

The audit record is written before the control decision is finalized, so that every verdict is logged regardless of the action taken.

7. Threat Model

Assets: the application of the check, the enforcement action, and the audit record.

Defended (in scope):

- T1. Post-hoc alteration or deletion of audit records.
- T2. Silent disabling or removal of the check by software, vendor, or operator.
- T3. Suppression of the enforcement gate's response to a verdict.

Not defended (out of scope):

- O1. **Incorrect judgment** by the model-dependent verifier — wrong attribution or score.
- O2. **Adversarial inputs** crafted so that an unsafe output attributes to a "safe" coordinate.
- O3. **Risks with no structural-coordinate signature**, outside HNS's expressible scope.
- O4. **Omission in uncertified deployments** — immutability prevents modifying the checker, not omitting it. The guarantees hold only where the hardware is in the path; in regulated contexts this can be a condition of conformity assessment, but it cannot otherwise be assumed.

8. The Performance Question (Open)

Pre- and post-inference structural checks add latency, and a hardware-mediated path may add more. This cost is unknown and potentially significant — in the worst case a serious handicap for throughput-sensitive systems. It is not a deferrable detail; it determines whether the architecture is practical, and it cannot be settled analytically. ROM maximizes immutability but is inflexible; FPGA enables measurement; ASIC could optimize steady-state cost if the approach proves worthwhile. The honest position is that we will not know whether the safety floor is worth its speed cost until we build and measure it (§10).

9. Relationship to Existing Work

Several ingredients already exist: hardware roots of trust (TPM, HSM, secure enclaves); runtime verification and monitors; and model-agnostic output guardrails and classifiers. Hardware immutability, physical enforcement, and external output checking are therefore not novel in themselves. The candidate axis of novelty — to be argued rather than asserted — is that the *content* placed under hardware enforcement and logging is a standardized, model-agnostic, governance-mapped structural schema (HNS-36) rather than an ad hoc or vendor-specific ruleset: a proposal to standardize *what* is enforced and recorded. Whether this adds practical value over existing approaches is an open question to be settled by evaluation.

10. Implementation Roadmap and Proof of Concept

10.1 Proof of concept

A minimal PoC — HNS-36 combined with EVA and an LLM over an N-turn reasoning session — should measure, against pre-defined criteria:

1. **Detection** — the rate at which the layer flags the intended class of structural deviations.
2. **Integrity** — that logs are demonstrably tamper-evident and the enforcement gate is non-disableable by software.
3. **Cost** — the added latency and throughput penalty across ROM and FPGA configurations.

10.2 Path forward

Item (2) and the integrity design are what the hardware claims to guarantee; (1) and (3) are the empirical questions that decide practical usefulness. Subsequent steps — engagement with standardization bodies and integration with conformity assessment — depend on these results, not the reverse.

11. Limitations

Stated plainly, as a condition of credibility for an expert reader:

- **Judgment correctness is out of scope** — the verifier is model-dependent; the layer secures the handling of a verdict, not its accuracy.
- **Coverage is bounded** — only HNS-expressible risks are addressed.
- **The guarantee is conditional** — it holds only where the hardware is in the enforcement path (§7, O4).
- **Performance cost is unquantified** — possibly prohibitive for some deployments (§8).
- **Novelty is unestablished** — differentiation from existing work (§9) is a hypothesis requiring evaluation.

12. Conclusion

Implementing HNS in hardware does not deliver complete or absolute AI safety, and this report does not claim it. Its significance is that it converts three safety properties — independence, model-agnosticism, and tamper-resistance — from vendor promises into architectural facts, each defined precisely in §3. Its effectiveness, within the bounded scope set out here, is a dependable, non-bypassable, independently auditable floor for structural safety checks: a layer that cannot be silently disabled and whose records cannot be altered, verifiable without reliance on vendor trust. Whether that floor is worth its performance cost, and whether its structural schema adds value over existing approaches, are empirical questions whose answer is a proof of concept measuring detection, integrity, and latency.